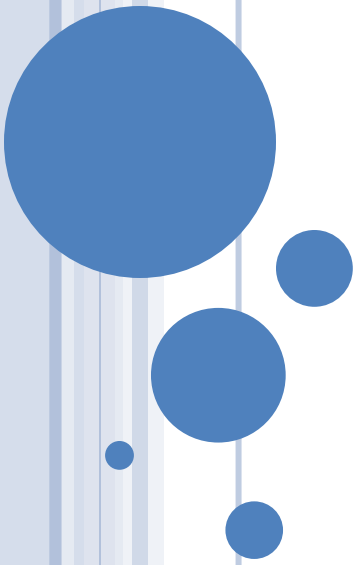


Теория автоматов и формальных языков

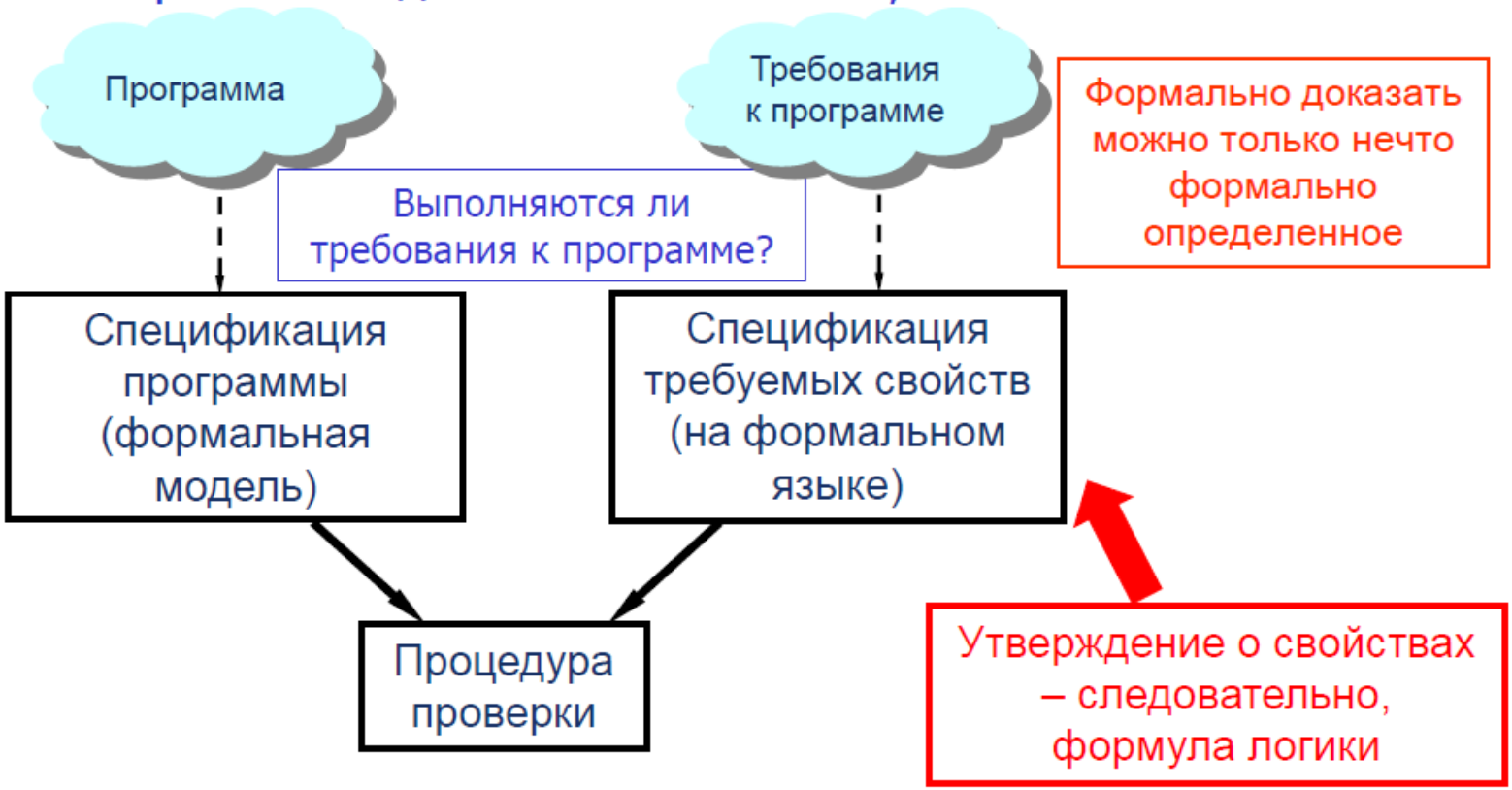
Метод Флойда верификации программ

Формальная постановка задачи верификации программ

Как **гарантировать** корректность программы?

Нужно сформулировать, что программа ДОЛЖНА делать, и проверить, делает ли она это

Формальное доказательство того, что в ПО нет ошибок

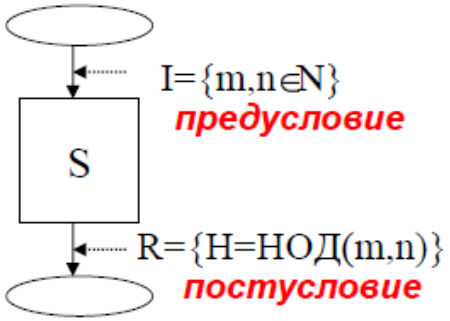


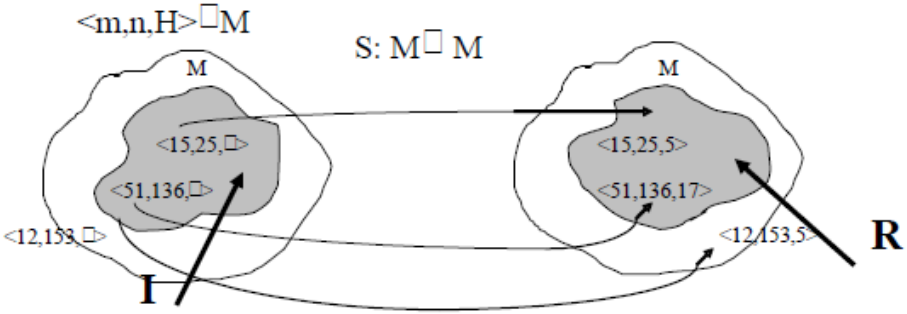
Спецификация программ обработки данных

Программу будем рассматривать, как преобразователь на множестве всех ее возможных ***состояний***

Состояние – набор значений переменных программы

Число состояний бесконечно

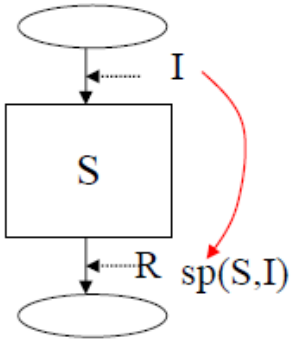




I – предусловие: предикат, который задает все такие значения входных переменных, для которых мы гарантируем правильную работу программы. Например, для программы извлечения квадратного корня $x \geq 0$

R – постусловие: задает соотношение между входом и выходом программы, т.е. правильный результат после работы программы

Корректность программы при вычисленном sp(S, I)



sp(S,I) – сильнейшее постусловие - утверждение, которому удовлетворяют программные переменные после завершения программы S, если они удовлетворяли предусловию I

Пусть сильнейшее постусловие sp построено

У нас есть программа и три предиката:

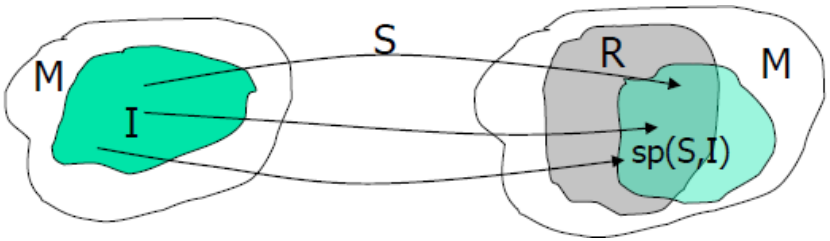
S - программа

I – предусловие - условие, которому должны удовлетворять программные переменные до выполнения программы

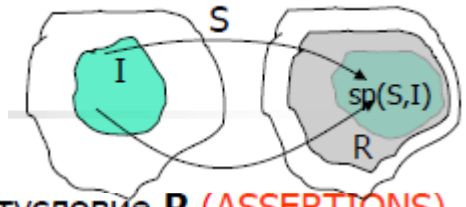
R – постусловие, которому должны удовлетворять значения переменных программы после выполнения программы

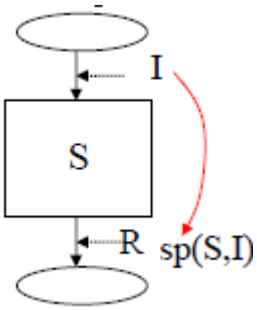
sp(S,I) – сильнейшее постусловие

Какое между ними должно быть соотношение, чтобы программа S была корректна?



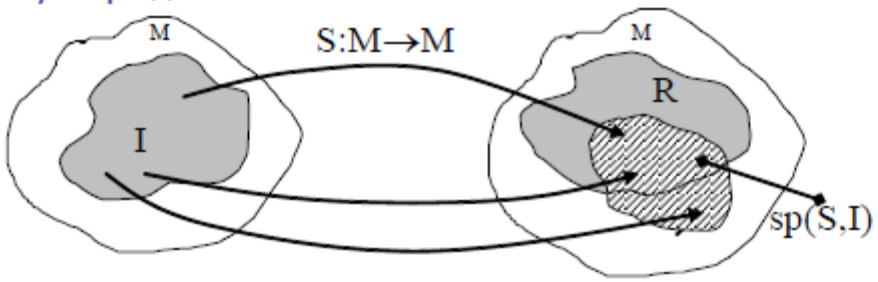
Триада Хоара: {I} S {R}





Пусть заданы S, предусловие **I** и постусловие **R** (ASSERTIONS)

Тройка (триада) Хоара {I} S {R} – *это утверждение*
если I *истинно до начала* S, ***то*** *после завершения* S
утверждение R станет *истинным*



Программа S (частично) корректна тогда и только тогда, когда sp(S,I) целиком лежит в R, т.е. когда формула sp(S,I)⇒R общезначима

Теорема. $\models\{I\}S\{R\} \Leftrightarrow sp(S, I) \Rightarrow R$

Верификация программы –задача логического вывода

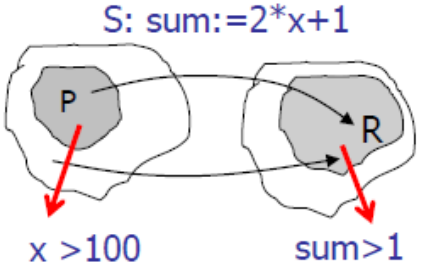
Слабейшее предусловие программы

Введено Эдсгером Дейкстрой в его книге “Дисциплина программирования”

P:: {??}
S: *sum* := 2*x + 1
R:: {*sum* > 1}

Каким должно быть *предусловие* оператора S, чтобы ПОСЛЕ выполнения S выполнялось бы *sum* > 1?

Пусть P={*x*>10}. Если x удовлетворяет P до выполнения S, то после выполнения S, *sum* удовлетворяет R.
Другое предусловие: {x>100}. Тоже подходит!
Какое предусловие выбрать? {x>10} или {x>100}?



Как найти максимально широкое предусловие P, такое, что из любого состояния, удовлетворяющего P, после выполнения S состояние будет удовлетворять R.

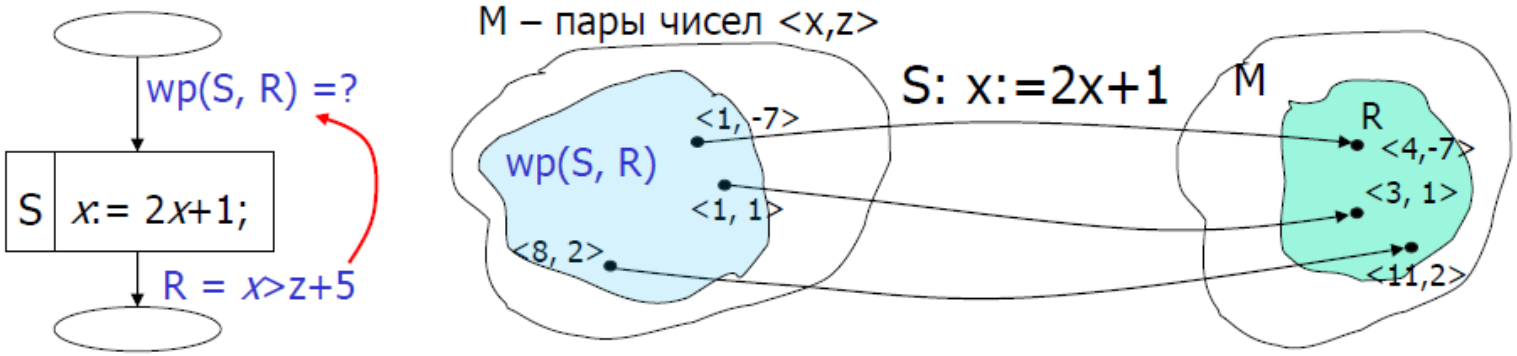
Наиболее широкое – значит слабейшее по ограничениям!

Дейкстра назвал его *weakrest precondition*, wp

Для оператора sum := 2*x + 1 и постусловия {sum >1} все предусловия {x>10}, {x > 50} и {x > 1000} правильны. Но ***слабейшее*** из всех предусловий - {x > 0}

Программа как преобразователь предикатов

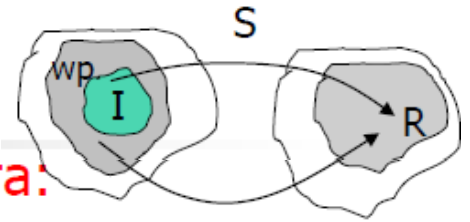
По постусловию R в конце программы S можно вычислить СЛАБЕЙШЕЕ ПРЕДусловие (weakest precondition), wp(S, R) – зависит от S и от R

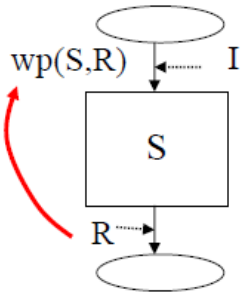


Определение. Слабейшее предусловие программы wp(S,I) задает множество **ВСЕХ** тех состояний программы S, из которых в результате работы S попадем в состояния, которые удовлетворяли предикату R по завершении S

***После** завершения* программы S значение *x удвоится и* возрастет на 1.
Если R = x > z+5 после выполнения S = x:=2x+1, то wp(S, R) = x>z/2+2

Корректность программы как преобразователя предикатов





Есть программа и три предиката:

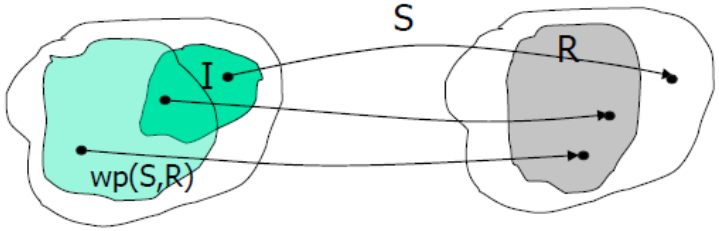
S - программа

I – предусловие, которому должны удовлетворять состояния программы до ее выполнения

R – постусловие, которому, должны удовлетворять состояния программы после ее выполнения

wp(S,R) – слабейшее предусловие, которому удовлетворяют **все** такие начальные состояния S, из которых после завершения S попадем в состояния, удовлетворяющие R

Какое между ними должно быть соотношение, чтобы программа S была корректна (т.е. из любого состояния, удовлетворяющего I, попали бы после выполнения S в состояние, удовлетворяющее R)?



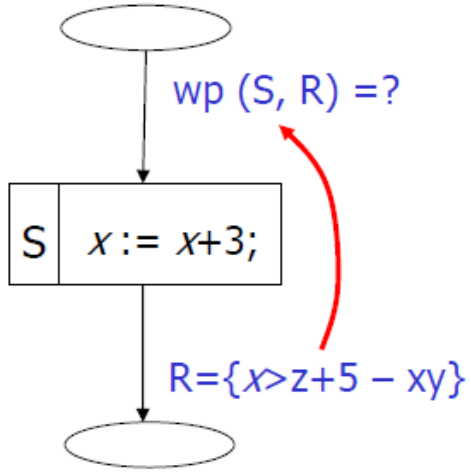
Теорема. {I}S{R} тогда и только тогда, когда I ⇒wp(S,R)

КАК НАЙТИ weakest precondition - wp(S,R)
для блоков присваивания и условия?



Слабейшее постусловие для оператора присваивания

Как *формально* определить правила преобразования *любого* предиката операторами языка?



Возьмем простейший оператор – оператор присваивания

Чему равен предикат

wp (x := x+3, x>z+5 – xy) =?

weakest precondition

программа

postcondition

условие, которому удовлетворяют **все** такие начальные состояния S, из которых после завершения S попадем в состояния, удовлетворяющие R

Вычисление слабейшего предусловия

wp(S,R) = ?
x:= x+3;
{R: *x*>z+5 – xy}

Неформально: ***После*** *завершения* программы *x* возрастет на 3. Для того, чтобы *после* завершения программы *x стало больше* z+5 – xy, нужно, чтобы *до начала программы* было

$x > z+2 - xy - 3y$

Пусть R(*x*) – предикат *после* выполнения оператора *x* := f(*x*). Обозначим *x'*- новое значение *x после завершения выполнения оператора x* := f(*x*). Тогда *x'* = f(*x*) (x'=f(x) - это не присваивание, это просто констатация факта: так связаны x' и x).

Предикат R выражен через новое значение *x*, т.е., это R(x'): x'>z+5-Xy

В начале программы R, выраженное *через переменную x'*, сохранится: wp(x'=f(x), R(x')) = R(*x'*) = { x′ >z+5 – x′y}

Но мы хотим, чтобы wp(S, R) был выражен через СТАРОЕ значение x

Как выразить R через старое значение x? Соотношение: **x' = f(x)**

wp(x:=f(x), R(x)) = R(f(*x*)) – слабейшее предусловие находится легко!

Пример: wp(x:=x+3, {*x*>z+5 – xy}) = {(*x*+3) > z+5–(x+3)y} ≡ {*x* >z+2–xy – 3y}

Школьный пример

"Из графика какой функции после его сдвига на одну единицу вправо и на две единицы вниз получится парабола $y = x^2 - 6x + 2$?"

Задачу можно сформулировать в наших терминах:

wp (S, R) =?
S1: $x := x + 1$;
S2: $y := y - 2$;
$R=\{y = x^2 - 6x + 2\}$



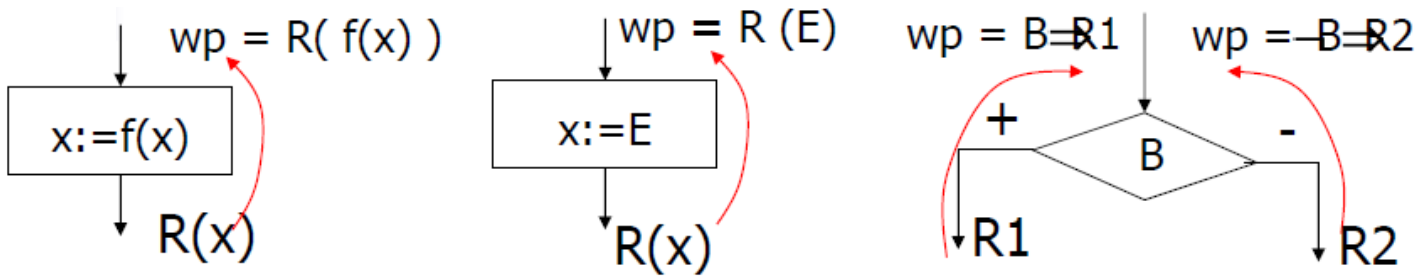
wp (S, R) = { wp ([S1; S2], R(x)) } ≡

сначала вместо y подставляем y-2,
а *потом* вместо x подставляем x+1

≡ $\{ (y - 2) = (x+1)^2 - 6(x+1) + 2 \}$
≡ $\{ y = x^2 - 4x - 1 \}$

Ответ: до сдвига был график параболы $y = x^2 - 4x + 1$

Формальная семантика операторов языка блок-схем



Для верификации программы с каждым ее оператором и с их комбинациями связываются преобразования предикатов, что является формальным выражением их свойств (их смысла, их семантики)

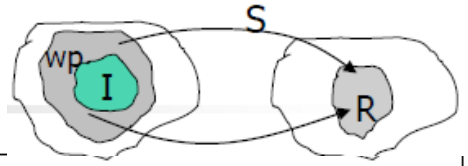
Формальная семантика операторов программы

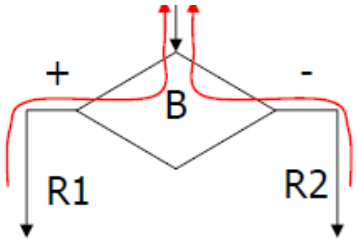
ЕСЛИ для любого предусловия I сможем вывести слабейшее предусловие wp(S, I), ТО понятно, как работает оператор S (известна формальная семантика S)

Такая семантика строится только для ограниченного множества операторов:

- на структурном языке программирования (фактически, блок-схемы)
- без использования адресной арифметики
- без подпрограмм
- без взаимодействия с окружением

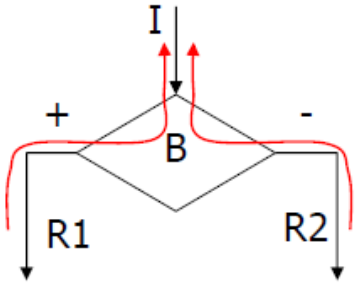
“Протаскивание” предиката через блок условия





Для того, чтобы состояния программы удовлетворяли утверждению R1, если B истинно, *ее состояния до начала выполнения программы должны удовлетворять предикату* $wp(B_{да}, R1) = B \Rightarrow R1$

Для того, чтобы состояния программы удовлетворяли утверждению R2, если B ложно, *ее состояния до начала выполнения программы должны удовлетворять предикату* $wp(B_{нет}, R2) = \neg B \Rightarrow R2$



$X \Rightarrow (Y \Rightarrow Z) = \neg X \vee \neg Y \vee Z = XY \Rightarrow Z$

Если до выполнения условного оператора B состояния программы удовлетворяют предикату I, то после выполнения оператора состояния программы будут удовлетворять:

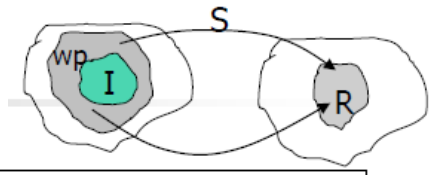
предикату R1 тогда и только тогда, когда

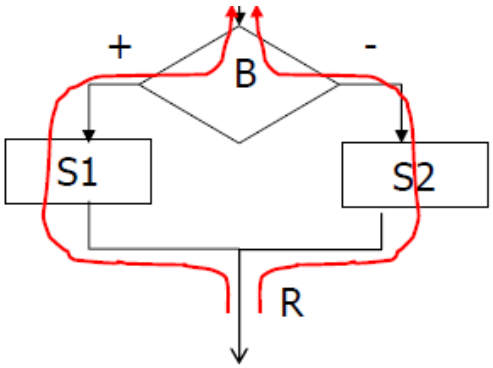
$I \Rightarrow wp(B_{да}, R1) \equiv I \Rightarrow (B \Rightarrow R1) \equiv I \wedge B \Rightarrow R1$

предикату R2 тогда и только тогда, когда

$I \Rightarrow wp(B_{нет}, R2) \equiv I \Rightarrow (\neg B \Rightarrow R2) \equiv I \wedge \neg B \Rightarrow R2$

Семантика оператора выбора





wp(if B then S1 else S2, R) =

(B ⇒wp(S1, R)) ∧ (¬B ⇒wp(S2, R))

Программа S частично корректна ттогда I ⇒wp(S,R)

I ⇒(B ⇒wp(S1, R)) ∧ (¬B ⇒wp(S2, R))

Или, что то же самое, ттогда: (I ∧ B ⇒wp(S1, R)) ∧ (I ∧ ¬B ⇒wp(S2, R))

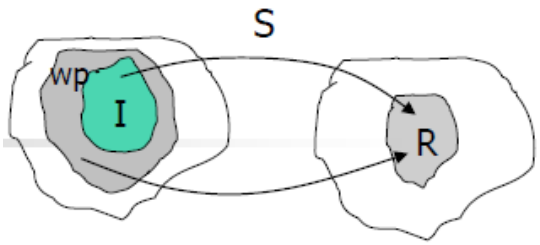
Поскольку X ⇒Y∧Z ≡ (X ⇒Y) ∧ (X ⇒Z), можно доказать две простые теоремы

Итак, программа if B then S1 else S2 частично корректна относительно I и R, если выполняются две теоремы:

T1: I ∧ B ⇒wp(S1, R) И

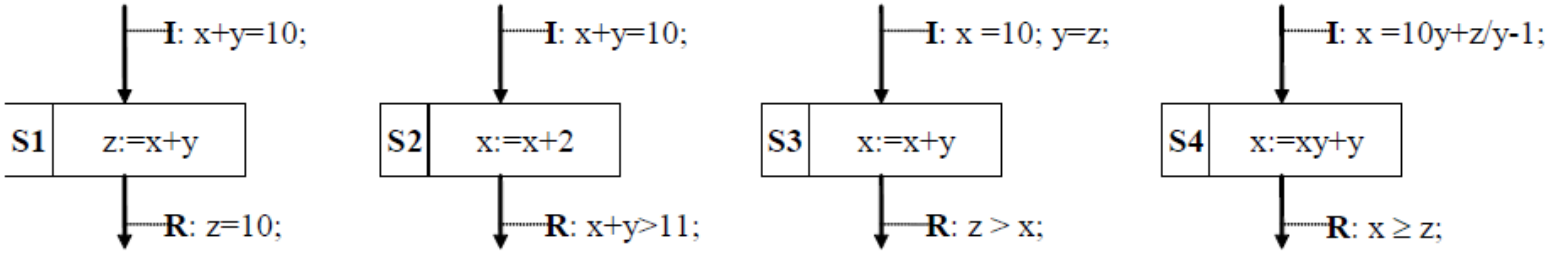
T2: I ∧ ¬B ⇒wp(S2, R)

Доказательство корректности простейших программ



$I \Rightarrow (?) wp(S, R)$

Примеры



Следует ли wp из предусловия?

S1: $wp(S1, R) \equiv (x+y=10)$; $x+y=10 \Rightarrow (?) (x+y=10)$ **ДА!** S1 корректна

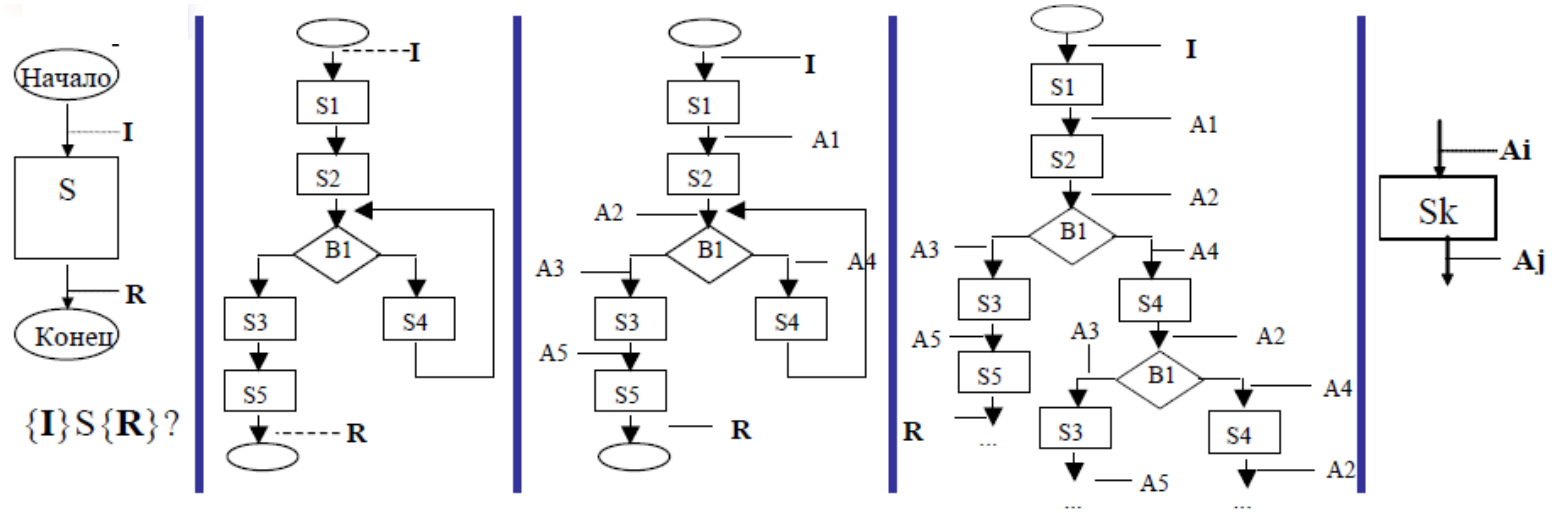
S2: $wp(S2, R) \equiv (x+2+y>11)$; $x+y=10 \Rightarrow (?) (x+2+y>11)$ **ДА!** S2 корректна

S3: $wp(S3, R) \equiv (z>x+y)$; $(x=10)\&(y=z) \not\Rightarrow (z>x+y)$ **НЕТ!** S3 некорректна

S4: $wp(S4, R) \equiv (xy+y>z)$; $x =10y+z/y-1 \Rightarrow (?) (xy+y>z)$ **ДА!** S4 корректна

wp строится очень просто!

Индуктивный метод Роберта Флойда доказательства частичной корректности программ (1967)



Для каждого блока Si истинность утверждения $\{A_i\}\ S_k\ \{A_j\}$ доказывается с помощью либо *strongest postcondition* либо *weakest precondition*

Теорема. *Пусть для каждой тройки $\{A_i\}\ S_k\ \{A_j\}$ доказана ее истинность. Тогда, если **I** истинно, то каждое утверждение A_r, встретившееся на пути вычислений (в том числе и R), истинно.*

По индукции ***если программа завершается*** (т.е. если **R когда-нибудь действительно** встретится на вычислении) то истинна тройка {**I**} S {**R**} – *если I истинно на входе в S, и S завершается, то на выходе истинно R*

Пример: корректность программы нахождения Max(x,y)

Программа S частично корректна, если I ⇒wp(S,R)

Доказательство программы Max:

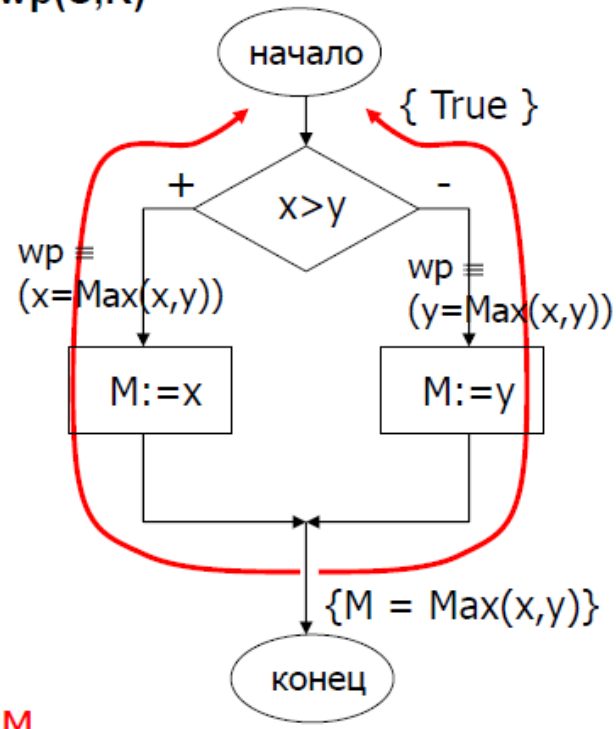
T1: True ∧ x>y ⇒x=Max(x, y) ≡

x > y ⇒x=Max(x, y)

T2: True ∧ ¬x>y ⇒y = Max(x, y) ≡

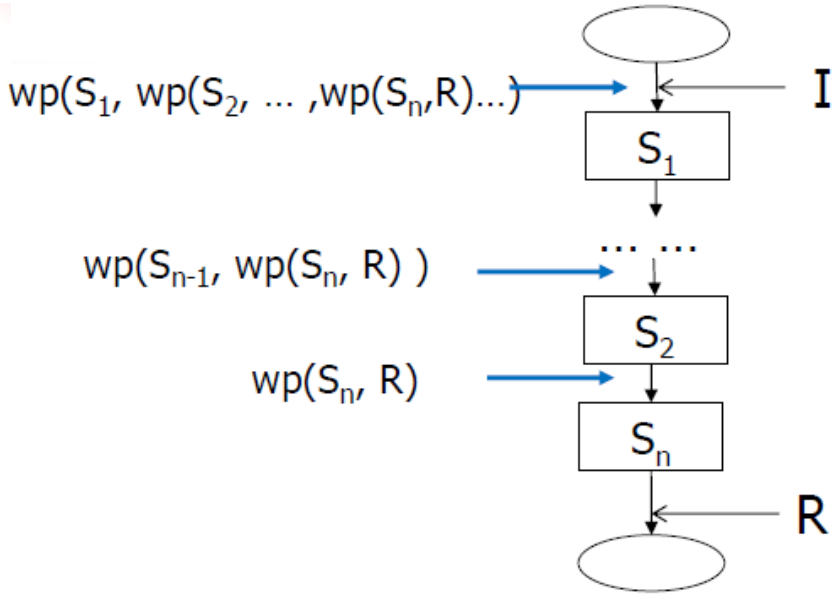
x ≤ y ⇒y=Max(x, y)

Но эти теоремы T1 и T2 и есть определение максимума двух элементов



Доказывать корректность программ с помощью wp легче!

Доказательство корректности ациклических программ: “протаскивание” предикатов



Дополнительных утверждений, кроме I и R, придумывать не нужно!

Для доказательства корректности программы, состоящей из последовательности операторов, нужно доказать ОДНУ теорему:

$$I \Rightarrow wp(S_1, wp(S_2, \ldots, wp(S_n, R) \ldots)\,)\,) \quad (*)$$

Программа S1;…;Sn корректна, iff формула (*) истинна

Пример: доказательство простой программы

Построить предусловие, при котором программа S корректна, т.е выполняет то, что от нее требуется (устанавливает x>y):

{wp: **?**}
x:=x+y;
y:=x*y
{ R: x > y }

Решение:

wp (S, R) =
= wp ([x:=x+y; y:=x*y], {x > y}) ≡

(*сначала* y заменяем на x*y, а потом x – на x+y)

≡ { x + y > (x+y)*y } – нужно рассмотреть три варианта
≡ { (x+y ≠ 0) ∧ (x+y > 0 ⇒ y < 1) ∧ (x+y < 0 ⇒ y >1) }

Пример построения wp для оператора выбора

wp([if y=0 then x :=0 else x := y+1], {x=y}) =

по определению wp оператора выбора

$(y=0 \Rightarrow wp([x:=0], \{x=y\}) \wedge (y\neq 0 \Rightarrow wp([x:=y+1], \{x=y\}) \equiv$

$(y=0 \Rightarrow 0=y) \wedge (y\neq 0 \Rightarrow y = y+1)) \equiv$

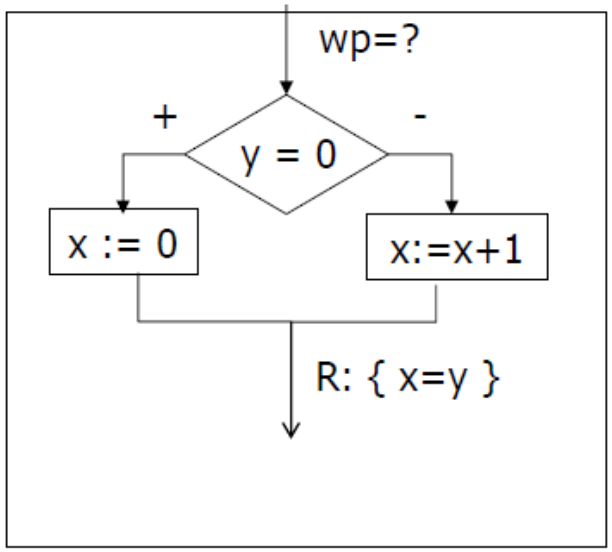
$True \wedge (y\neq 0 \Rightarrow y=y+1) \equiv$

$(y\neq 0 \Rightarrow y=y+1) \equiv$

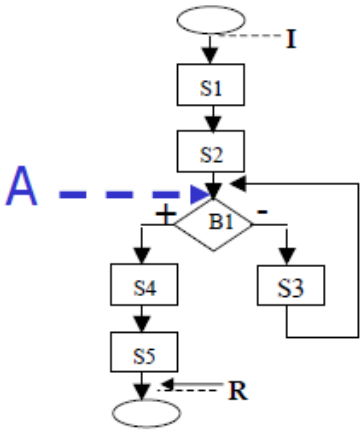
$(y\neq 0 \Rightarrow False) \equiv$

$\neg(y\neq 0) \equiv$

$(y=0)$



Инварианты циклов: утверждения о поведении циклов



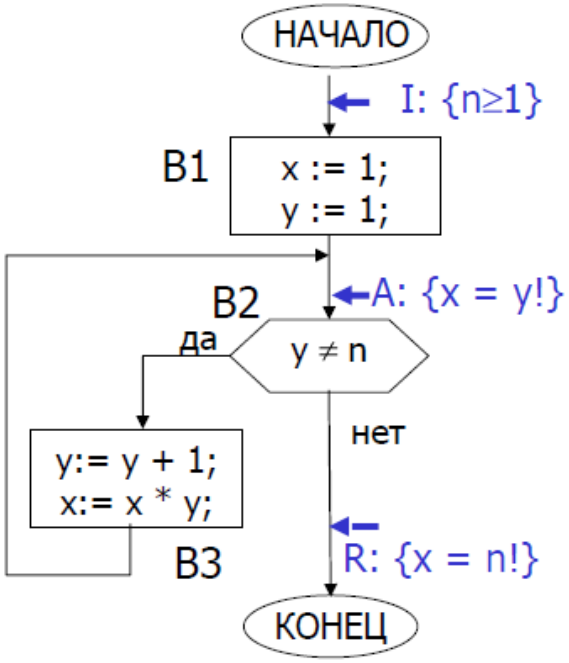
Для доказательства корректности нет необходимости придумывать все утверждения, после каждого блока

Достаточно придумать только инварианты – по одному такому утверждению в каждом цикле, которые остаются истинными при каждом прохождении цикла

Если инвариант цикла A придуман, то доказательство циклической программы сводится к доказательству трех теорем:
T1: $I \Rightarrow wp([s1; s2], A);$
T2: $A \Rightarrow wp([B1-; s3], A);$
T3: $A \Rightarrow wp([B1+; s4; s5], R);$

Если невозможно доказать корректность программы, то или программа неверна, или неверно придуманы утверждения на стрелках программы (инварианты циклов)

Доказательство частичной корректности программы вычисления факториала



$X \Rightarrow (Y \Rightarrow Z) = \neg X \vee \neg Y \vee Z = XY \Rightarrow Z$

Доказательство теорем

T1: $I \Rightarrow (?)\ wp(B1, A)$

$wp(B1, A) = (1 = 1!) = True$

$(n \geq 1) \Rightarrow (?)\ True$ ДА! (True - следствие всего!)

T2: $A \Rightarrow (?)\ wp([B2да; B3], A)$

$wp([B2да; B3], A) = B2да \Rightarrow wp(B3, A)$
$wp(B3, A) = (x*y = (y+1)!)$

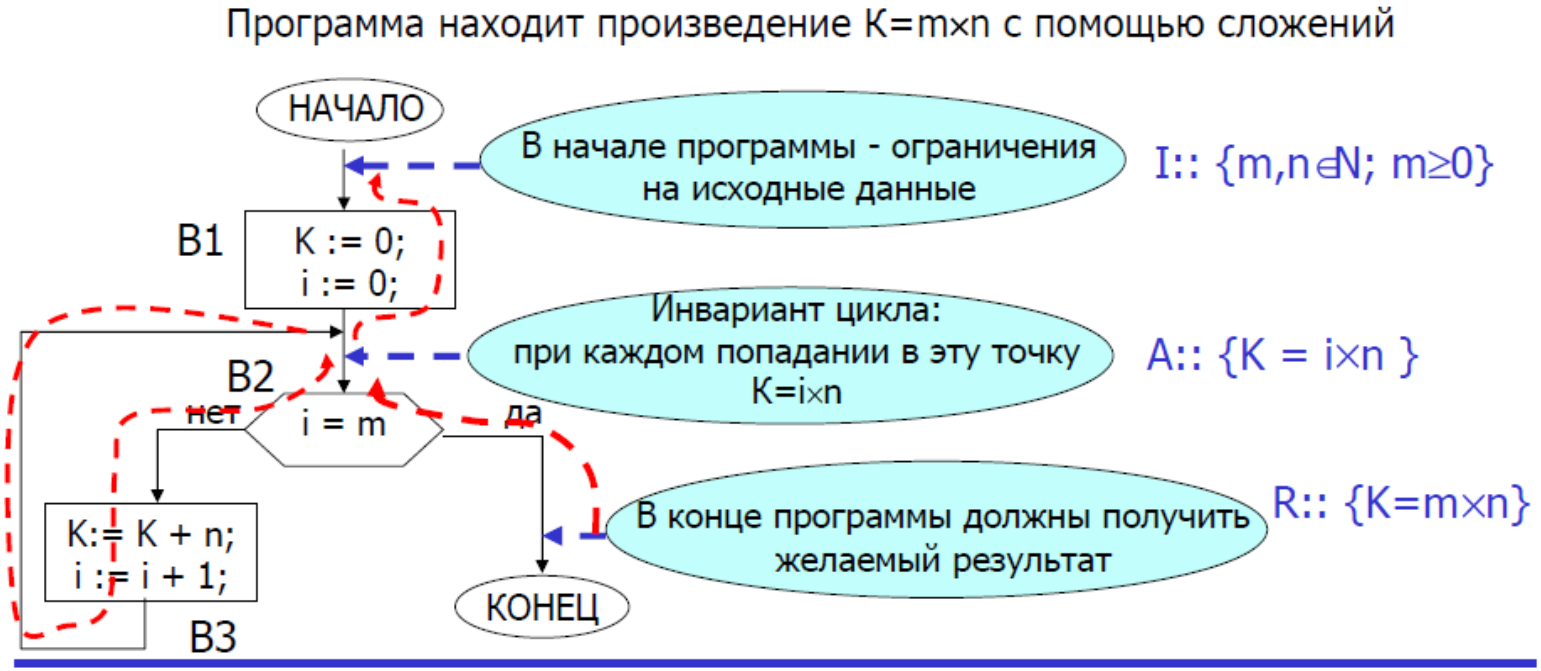
$(x = y!) \Rightarrow (?)\ ((y \neq n) \Rightarrow (x*y = (y+1)!)) =$
$(x = y!) \wedge (y \neq n) \Rightarrow (?)\ (x*(y+1) = (y+1)!)$ ДА!

T3: $A \Rightarrow (?)\ wp(B2нет, R)$

$wp(B2нет, R) = \neg(y \neq n) \Rightarrow (x = n!) = (y = n) \Rightarrow (x = n!)$

$(x = y!) \wedge (y = n) \Rightarrow (?)\ (x = n!)$ ДА!

Пример доказательства корректности с WP



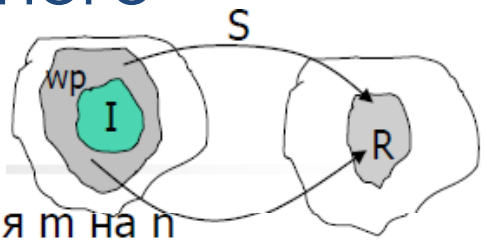
Доказательство корректности программы требует доказательства трех теорем

T1: $I \Rightarrow wp([B1], A)$

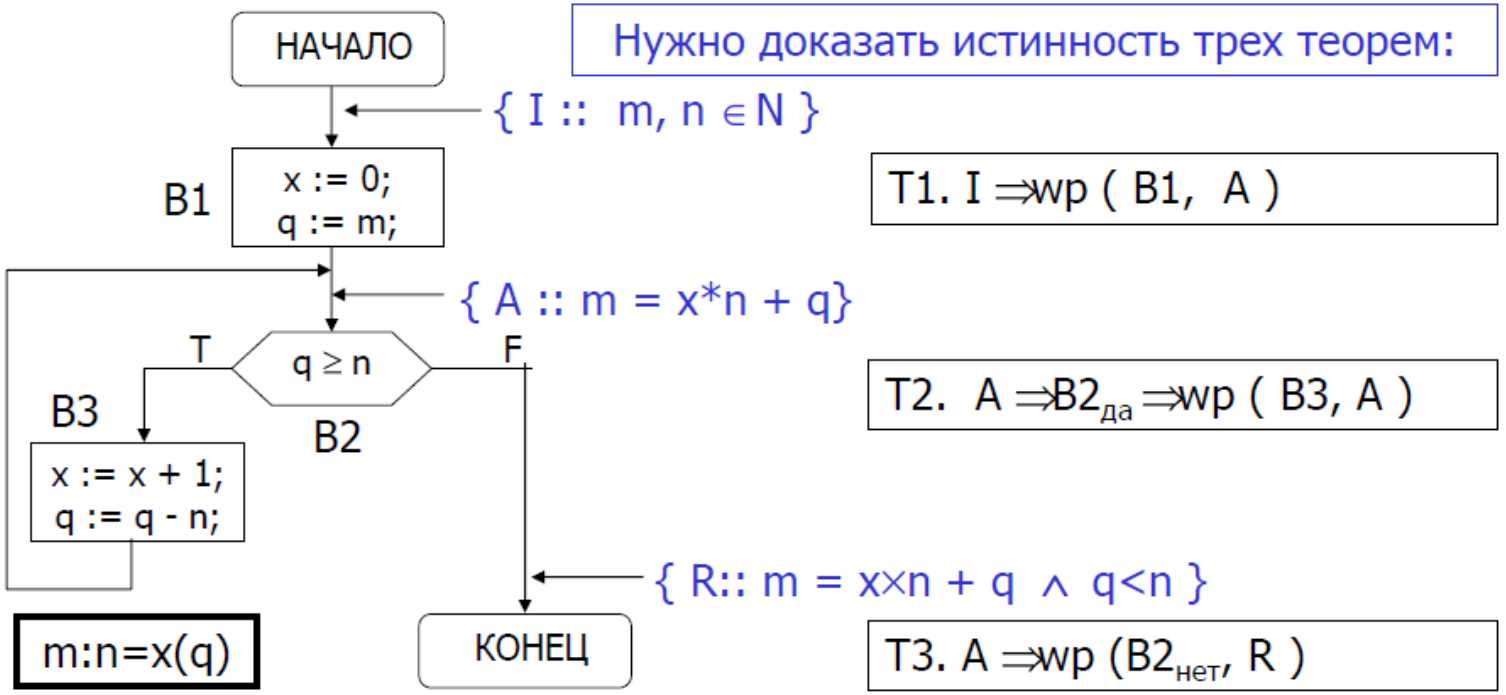
T2: $A \wedge B2нет \Rightarrow wp(B3, A)$

T3: $A \Rightarrow wp(B2да, R)$

Пример: программа целочисленного деления



Программа находит частное x и остаток q от деления m на n



Нужно доказать истинность трех теорем:

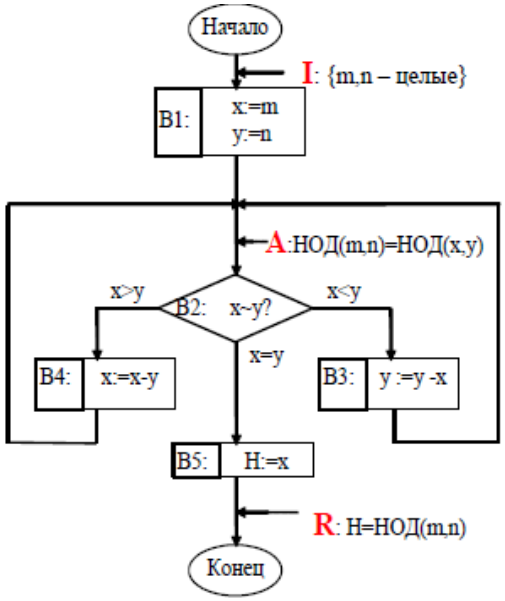
T1. $I \Rightarrow wp(B1, A)$

T2. $A \Rightarrow B2_{да} \Rightarrow wp(B3, A)$

T3. $A \Rightarrow wp(B2_{нет}, R)$

- После того, как докажем все три теоремы T1, T2 и T3, можем сделать вывод, что программа частично корректна
- Но программа не тотально корректна. При n=0 она не завершается

Доказательство частичной корректности программы нахождения НОД



Программа снабжается кроме **I** и **R** ***только одним*** дополнительным утверждением: ***инвариантом цикла*** A

Нужно доказать истинность четырех теорем:

T1. $I \Rightarrow wp(B1, \mathbf{A})$

T2. $\mathbf{A} \wedge B2_{x<y} \Rightarrow (?)\ wp(B3, A)$

T3. $\mathbf{A} \wedge B2_{x>y} \Rightarrow (?)\ wp(B4, A)$

T4. $\mathbf{A} \wedge B2_{x=y} \Rightarrow (?)\ wp(B5, R)$

или:

T1. m,n – целые $\Rightarrow$ НОД(m,n) = НОД(m,n)

T2. (НОД(m,n)=НОД(x,y)) $\wedge$ x<y $\Rightarrow$ НОД(m,n)=НОД(x, y-x)

T3. (НОД(m,n)=НОД(x,y)) $\wedge$ x>y $\Rightarrow$ НОД(m,n)=НОД(x-y, y)

T4. (НОД(m,n)=НОД(x,y)) $\wedge$ x=y $\Rightarrow$ x = НОД(m,n))

Эти четыре теоремы полностью определяют НОД

Программа ***частично корректна:*** при отрицательных m либо n программа входит в бесконечный цикл

Формулировка утверждений требует проникновения в суть алгоритма

Практическое использование дедуктивной верификации



Дедуктивный метод верификации программ (HOL, Isabelle , ...)



- Спецификация – на языке программирования
  - Для операторов языка определена ***семантика преобразования предикатов***
- Спецификация требований
  - Описываются как пара вход-выходных утверждений
  - Добавляются инварианты циклов и промежуточные утверждения, которые отражают идею алгоритма
- Процедура проверки
  - Не очень эффективна (только частично автоматизированная) система доказательства теорем, работает с помощью пользователя)

Верификация методом Флойда – процесс вывода теорем (называется дедуктивной верификацией)

- Семантика языка (правила того, как конструкции языка преобразуют предикаты) может быть представлена для многих конструкций языка:
  - сложные структуры данных: целые, списки, деревья и т.п.;
  - сложные программные конструкции (рекурсия, циклы)

  Это позволяет представить и проверить тонкие свойства программ
- НО: достижение этой точности имеет свою цену: трудно предсказать, какой объем ресурсов (времени, памяти) может потребоваться для верификации конкретной программы: в общем случае, проблема доказательства свойств программ, работающих с бесконечными структурами, алгоритмически неразрешима (следствие неразрешимости логики предикатов)



Пример верификации (с использованием Theorem Prover Isabelle)

Сообщение СМИ 2009 г.

- Research Centre of Excellence компании NISTA (Australia) объявил о завершении работы по формальному доказательству корректности ядра ОС с помощью интерактивной системы доказательства теорем Isabelle
- Доказанная ОС - The Secure Embedded L4 (seL4) microkernel содержит 6,500 строк C code. Было формально доказано 10,000 промежуточных теорем, доказательство заняло 200,000 строк
- Это результат 4-х летнего труда группы из 12 исследователей NISTA под руководством Dr Klein, PhD студентов и нескольких других работников
- The proof uncovered 160 bugs in the C code of the seL4 kernel
- Этот экстраординарный результат открывает путь к разработке нового поколения ПО с беспрецедентным уровнем надежности. Это одно из самых длинных из когда-либо выполненных формальных доказательств с помощью формальных средств theorem-proving

Оценка этого результата

Построение доказательства ядра ОС secure embedded L4 потребовало ~ 60 человеко-лет работы (~750 чел.месяцев),
10 строк кода – 1 чел-месяц работы верификатора

Сложность доказательства в 30 раз выше, чем сложность кода;
одна строка кода - ~ 1 страница доказательства

Это согласуется с высказыванием Allen Emerson (U.Texas, Austin) о дедуктивной верификации: "*мы писали 15-страничный отчет о том, что программа на полстраницы корректна*"

"Формальные доказательства в форме логических выводов – необозримы"



Дедуктивная верификация – непростое дело

Из описания системы доказательства теорем PVS [Owre, S. et.al. PVS: Combining Specification, Proof Checking, and Model Checking //*LNCS* 1102, 1996]:

"PVS is a large and complex system and it takes a long while to learn to use it effectively. You should be prepared to invest six months to become a moderately skilled user ... ".

Вывод:

пока нельзя говорить о широком использовании дедуктивного метода верификации Флойда-Хоара (пред- и постусловий)!
Можно доказывать лишь программы небольшого размера



Современное состояние индуктивной верификации

- Существуют интерактивные верификаторы (theorem provers), например, Isabelle (Cambridge), PVS(Stanford), HOL
- Программист в свою программу в любом месте программы может записать в качестве комментария Assert – предикат (фактически, условное выражение), чтобы помочь в спецификации и понимании программы, чтобы отметить, что он думает, что всегда, когда управление проходит через эту точку программы, этот предикат истинен. Иногда такие утверждения называют ассертами (инвариант)
- В некоторых языках программирования Assert – это стандартный оператор, фактически вызывающий прерывание и переход на обработку исключительных ситуаций, если тело этого оператора (условное выражение) не вычисляется как true, когда программа проходит эту точку во время ее выполнения
- Контрактное программирование использует предусловие и постусловие для того, чтобы строго формально определить спецификацию – задание на разработку программы. В контрактном программировании используются:
  - *preconditions*: логические условия, которые должны быть выполнены при вызовах операций (обязанность вызывающего эту операцию)
  - *postconditions*: логические условия, выполнение которых гарантировано после завершения операции (использующему эту операцию)
- Такие спецификации называются "контрактами" в соответствии с *концептуальной метафорой* условий и ответственностей в бизнес-контрактах.

Защитное программирование: выполнение аннотированных программ

```
{I: n≥1}
x:=1;
y:=1;
{A: x=y! ∧ y≤n }
   while (y≠n)
      do
         y:=y+1;
         x:=x*y
      od
{ R: x=n!}
```

- Во многих языках программирования в программы могут быть добавлены аннотации (assertions) – "защитное программирование"
- Если утверждения (аннотации) записаны на языке программирования и могут быть вычислены, то при выполнении программы каждая аннотация проверяется на текущих значениях программных переменных, и в случае, если аннотация вычисляется как false, генерируется прерывание "ОШИБКА ВРЕМЕНИ ВЫПОЛНЕНИЯ ПРОГРАММЫ"



Разработка программ, корректных по построению



Идеи Эдсгера Дейкстры о разработке программ. Correctness by construction

- Эдсгер Дейкстра, великий голландский теоретик в области информатики, выдвинул следующую идею для разработки корректных программ:

вместо того, чтобы программу разрабатывать, а ПОТОМ пытаться доказать ее правильность, программу нужно сразу строить корректной

- ***Дейкстра***: Программа должна выводиться строго логически из точно сформулированной спецификации, а а НЕ исходя из опыта и общих соображений



В книге "Дисциплина программирования" Э.Дейкстры приведено множество изящных примеров такого подхода к разработке

"Этот труд, несомненно, будет признан одним из выдающихся достижений в разработке новой интеллектуальной дисциплины – программирования"

Guarded Commands – охраняемые команды

Предложены Э. Дейкстрой, использованы и в других приложениях, например, в языке Холзманна Promela (Protocol Meta Language)

Пример: Алгоритм Эвклида нахождения НОД

```
x, y := m, n
do
    [ ] x>y →x:=x-y
    [ ] y>x →y:=y-x
od
print x
```

Guarded Command: G →Op,

Присваивание: x, y, z := E1,E2,E3

Условный оператор:

```
if
   [ ] <command1>
   [ ] …
   [ ] <command n>
fi
```

Оператор цикла:

```
do
   [ ] <command1>
   [ ]        …
   [ ] <command n>
od
```

Недетерминизм в алгоритмах

В условном операторе при истинности нескольких защит выбирается случайно одна любая. Если все защиты ложны, то это ошибка

Оператор цикла завершается, если ни одна из защит не является истинной

Э.Дейкстра о недетерминизме в алгоритмах: “*оказывается, что именно недетерминированная природа защиты в операторах выбора и цикла позволяет естественным образом формализовать семантику этих операторов*”

Guarded Command: G → Op,

Присваивание: x, y, z := E1,E2,E3

Условный оператор:

```
if
   [ ] <command1>
   [ ]  …
   [ ] <command n>
fi
```

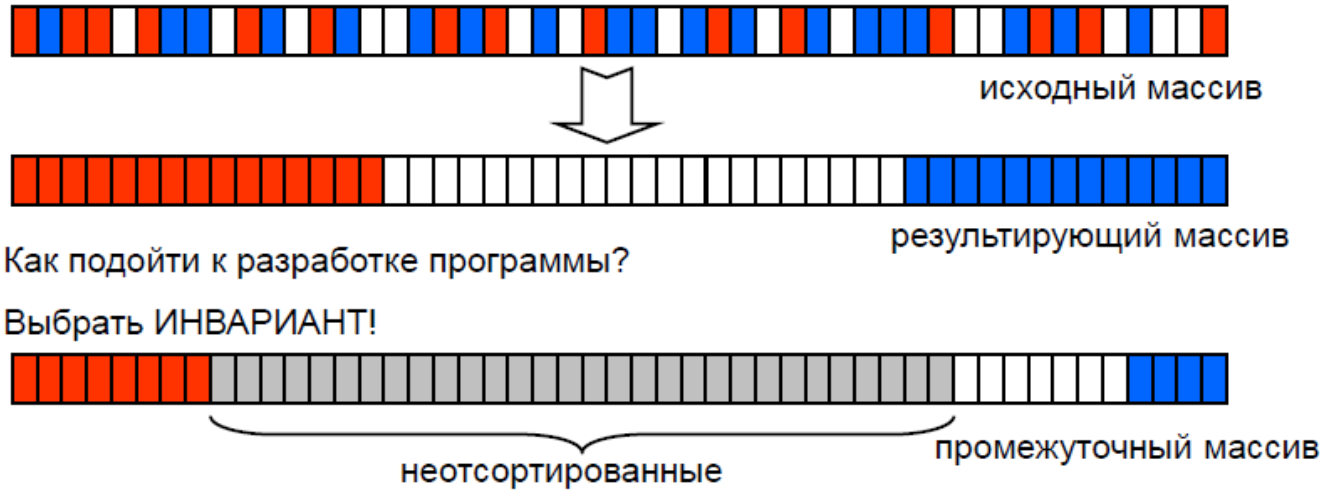
Оператор цикла:

```
do
   [ ] <command1>
   [ ]        …
   [ ] <command n>
od
```

Разработка корректных программ (Э.Дейкстра)



"Голландский национальный флаг": построить программу сортировки массива из трех типов объектов – "красных", "белых" и "синих", которые должны в результате стоять именно в этом порядке. Программа должна справляться со всеми случаями вырождения: отсутствием одного или нескольких цветов



Крайними состояниями инварианта являются **исходный и результирующий массивы**

Формализация инварианта



A ≡ [∀i: (1≤i<к)] M[i] - красный
& [∀i: (б<i≤с)] M[i] - белый
& [∀i: (с<i≤N)] M[i] - синий
& [∀i: (к≤i≤б)] M[i] - неотсортированный
& (к≥1) & (б≤с) & (с≤N)

Очевидно, что ЕСЛИ:
A & (к>б) , ТО массив M отсортирован
(массив неотсортированных пуст)

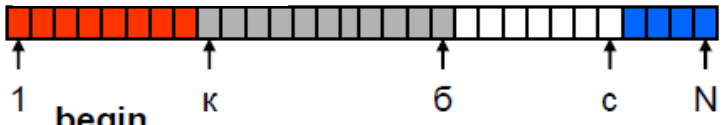
Разработка программы сводится к построению цикла, который уменьшает зону неотсортированных элементов, но при этом сохраняет инвариант

```
begin
 I = { [∀i: (1≤i≤N) ] M[i] - неотсортированные}
  к,б,с:=1,N,N;
  /* инвариант цикла  A */
do
  <пока зона не отсортированных непуста> ⇒
<уменьшить зону неотсортированных, не меняя A>
od
  { R≡ массив M отсортирован }
end
```

Программа частично корректна, если
1) из I, преобразованного оператором
k,б,с := 1,N,N следует A,
2) A – инвариант цикла,
3) из A следует R по завершении цикла

Программа тотально корректна, если при этом зона неотсортированных уменьшается на каждом шаге цикла

Разработка программы, корректной по построению



A ≡ [∀i: (1≤i<к)] M[i] - красный
& [∀i: (б<i≤с)] M[i] - белый
& [∀i: (с<i≤N)] M[i] - синий
& [∀i: (к≤i≤б)] M[i] - неотсортированный
& (к≥1) & (б≤с) & (с≤N)

begin
{ [∀i: (1≤i≤N)] M[i] – неотсортированный }
к,б,с:=1,N,N;
{A ≡ инвариант цикла}
do
к ≤б →
if [ ] M[б] = белый →б:=б-1;
[ ] M[б] = красный →поменять местами (M[б], M[к]); к:=к+1;
[ ] M[б] = синий →поменять местами (M[б], M[с]); с, б:=с-1, б-1;
fi
od
{R≡массив M отсортирован}
end

Для верификации программы нужно доказать:
1. sp (S0, I) ⇒A или I ⇒wp(S0, A)
2. A&¬G⇒R, т.е. A&(к<б) ⇒R
3. sp(Si, A&G&Gi)⇒A или A⇒wp(Si, A&G&Gi)
4. ***Завершение программы***: *нужно доказать, что **б-к** уменьшается на каждом шаге цикла*

Охраняемые команды Дейкстры:

begin
{ I }
S0
{A}
do
G →
if [ ] G1 →S1;
[ ] G2 →S2;
[ ] G3 →S3
fi
od
{R}
end

Метод Флойда и тестирование программ



Тестирование с использованием аннотированных программ

Если Pre-/Post-Conditions включены в качестве спецификации в программу, то:

• Для выполнения тестирования:

– Проверьте, что входной набор переменных удовлетворяет предусловию

– Погоните тест

– Проверьте, что результаты теста удовлетворяют постусловию

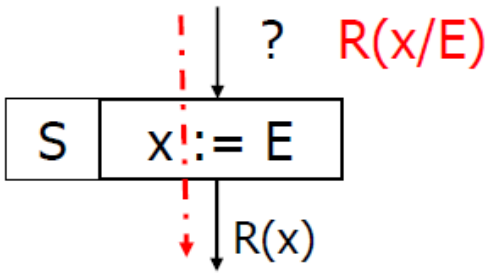
• Этот подход помогает прогонять тесты, но не помогает составлять их.
Полезно при рандомизированном тестировании – использовании случайных входов в программу

Например, известный Fuzz эксперимент на утилитах Unix

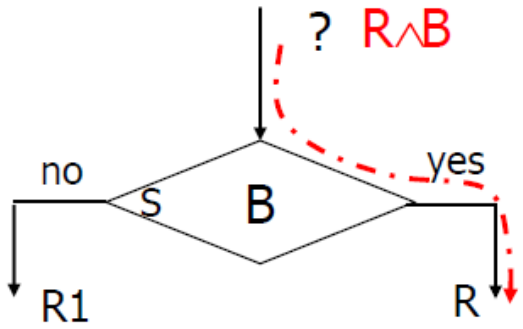


Использование weakest precondition для генерации тестов

Проблема: как определить такие начальные значения переменных программы, при которых вычисления пройдут точно по заданному пути?

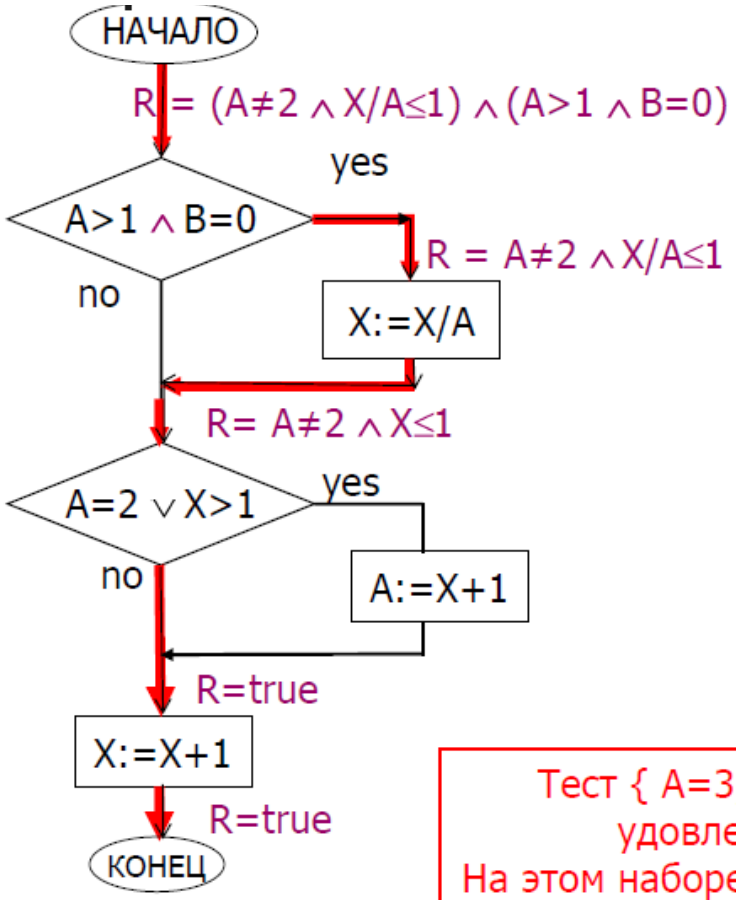


Для того, чтобы вычисления прошли через блок S :: "x :=E", **И** после выхода из блока S состояния программы удовлетворяли предикату R(x), достаточно, чтобы до входа в блок S состояния программы удовлетворяли предикату wp(R,S), т.е. R(x/E)



Для того, чтобы вычисления прошли через условный блок S через условие B, и после выхода из блока S состояния программы удовлетворяли предикату R, нужно, чтобы до входа в блок S состояния программы удовлетворяли предикату R∧B

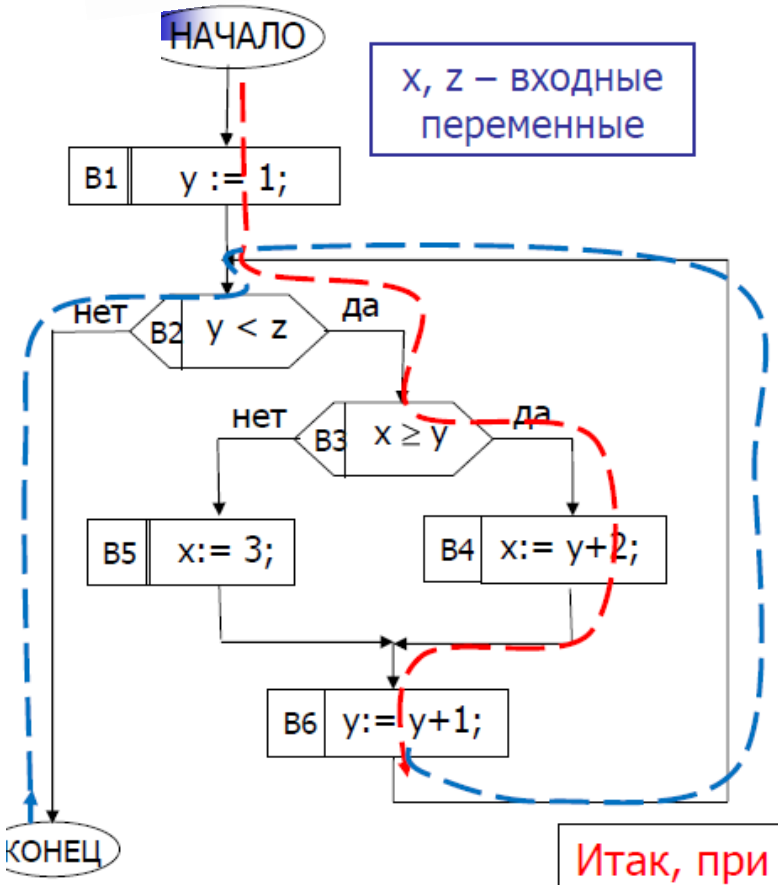
Генерация теста, проходящего в программе по заданному пути



- Выберем конкретный путь в программе
- Положим R:= true в конце пути
- Проходим путь в обратном порядке
- При прохождении присваивания x:=E, заменим все вхождения x в R на E; R(x) := R(E).
- При прохождении блока выбора по условию B, "no", R заменим: R:= R ∧ ¬B.
- При прохождении блока выбора по условию B, "yes", R заменим на R ∧ B; R:= R ∧ B.

Тест { A=3, B=0, X=2 } – значения переменных, удовлетворяющих R на входной стрелке.
На этом наборе вычисление пройдет по выбранному пути

Использование weakest precondition для генерации тестов



Например, путь
B1; B2да; B3да; B4; B6; B2нет

- Полагаем R=true
- "Протаскиваем" по обратному пути:

true -> B2нет ->
true ∧ ¬(y<z) = y≥z -> B6 ->
(y+1≥z) -> B4 ->
(y+1≥z)∧(x=y+2) -> B3да ->
(y+1≥z)∧(x=y+2)∧(x≥y) -> B2да ->
(y+1≥z)∧(x=y+2)∧(x≥y)∧(y<z) -> B1 ->
(1+1≥z)∧(x=1+2)∧(x≥1)∧(1<z) =
(1<z≤2)∧(1≤ x=3)

- Неравенству удовлетворяют:

x=3; z=2

Итак, при x=3, z=2 вычисление пройдет по заданному пути в программе

Заключение

- Тестирование – проверка работы программы на ее конкретных входных состояниях. Тестированием нельзя доказать корректность программы
- Программа в понимании Флойда, Хоара и Дейкстры является математическим объектом, и из этого следует, что корректность программы можно свести к формальным математическим операциям
- Предикат (аннотация, ASSERTION) описывает бесконечное множество возможных состояний. Связывая Assertion с конкретным местом в программе утверждаем ИНВАРИАНТ – при любом выполнении программы ее состояние в этом месте будет удовлетворять этому утверждению. Большинство современных ЯВУ содержат метод assert, который проверяет истинность утверждения во время выполнения и является основой “Защитного программирования”
- Рассматривая программу как преобразователь предикатов, можем рассуждать о корректности программы
- Доказательство правильности алгоритмов методом Флойда-Хоара сводится к расстановке утверждений в теле программы и доказательству теорем логики первого порядка, полученных из преобразованных предикатов в некоторой дедуктивной аксиоматической теории
- Спецификация программы – это пара предикатов <входной-выходной> для каждого блока. Как минимум, дополнительно требуется один предикат в каждом цикле (инвариант цикла) вдобавок к начальному и заключительному. **Формулировка утверждений требует проникновения в существо, основные идеи алгоритма, часто требует полного понимания (ultimate understanding) программы**
- Формальная семантика языка задается сильнейшим постусловием или слабейшим предусловием для каждого типа операторов и правилами вывода



Заключение

- Индуктивный метод Флойда и аксиоматический метод Хоара доказывают только частичную корректность программы; полная (total) корректность требует дополнительного доказательства завершаемости
- Метод Хоара – это формализация метода Флойда: сведение доказательства к формальной модели исчисления с аксиомами и правилами вывода, т.е. к аксиоматической теории. **Метод не может быть полностью автоматизирован из-за неразрешимости логики предикатов. Иными словами, не гарантируется завершение процесса доказательства**
- Реальное доказательство требует “ручного сопровождения” – формулировки промежуточных лемм и теорем (в реальных системах – многих тысяч их)
- Сегодня есть мощные инструменты, **частично** автоматизирующие доказательство (например, HOL(Higher Order Logic), Isabelle и PVS (Prototype Verification System))

Примеры использования системы верификации Isabelle :

- в 1996 г. доказана корректность алгоритма деления с плавающей точкой в процессоре AMD5K86. Это потребовало введения 1600 определений и лемм
- в 2005 г. доказана корректность алгоритма нормализации BDD
- в 2009 г. доказано ядро ОС seL4 из 6.5 тыс строк, 10 тыс теорем, ~80 чел лет
- В книге “Дисциплина программирования” Э.Дейкстра предложил методику разработки программ совместно с доказательством их корректности, т.е. метод разработки алгоритмов, корректных по построению. Этот метод пока не нашел реализации в технологии разработки программ



Нужно ли доказывать правильность программы?

- В случае тривиальных программ их доказательство - обременительная формальность доказательства очевидных вещей. Доказательство отнимает гораздо больше времени и труда, чем сама разработка, которая обычно базируется на очевидных положениях
- Не требуют сложной верификации бытовые программы, от которых мало что зависит
- В области параллельных программ ситуация не такая простая. Параллельные программы непонятны, даже если каждая ветвь их понятна. Кроме того, параллельные программы плохо поддаются тестированию
- **Другим классом программ, требующим верификации, являются программы, в которых цена ошибки весьма высока, например, если она превышает стоимость разработки программы.** Это относится к банковским и биллинговым программам, которые оперируют большими суммами. В них число условий, ограничений и ветвей огромно, они обычно трудны для тестирования и понимания
- Другая область, в которой важна верификация – системы управления, встроенные бортовые системы медицинской техники, космических систем, авиации, транспорта, ...
- На идеях Флойда и Хоара основана современная технология программирования по контракту (см., например: ttp://www.rsdn.ru/article/design/Code_Contracts.xml)



Что нужно помнить про верификацию программ методом Флойда-Хоара

- Этим подходом можно доказывать программы с бесконечным числом состояний (значений программных переменных)
- **Требования к программе задаются формулами логики предикатов**
- Проверка корректности программ основана на доказательстве теорем
- **Для доказательства используются лишь некоторые выбранные точки программы – задаются условия входа, выхода и инварианты циклов**
- В процессе верификации необходимо обязательное участие человека – полная автоматизация процесса доказательства **НЕВОЗМОЖНА**
- Формальное доказательство теорем требует глубоких знаний в этой области
- Доказательство даже небольших программ требует больших усилий, в десятки раз большие, чем разработка программ
- **Теоретические основы этого подхода довольно сложны**
- Можно ли гарантировать корректность самой системы доказательства теорем?



СПАСИБО
ЗА ВНИМАНИЕ